FRANKFURT UNIVERSITY OF APPLIED SCIENCES Fachbereich 2: Informatik Wahlpflichtmodul-Labor
Wintersemester 2025/26

PROJEKTBERICHT

THEMA: Smart Trash Bin

(Internet of Things – Smart Sensor Systems)

Eingereicht durch: Kaan Degirmenci (Matrikelnr: 1402131)

E-Mail: kaan.degirmenci@stud.fra-uas.de

Betreuer: Prof. Dr. Bergbauer

[1]**Datum:** 24.12.2025

1

Web: https://smarttrashcan-72fef.web.app

INHALTSVERZEICHNIS

Management Summary

1.Einleitung

2.Vorstellung der Hardware

2.1 Mikrocontroller

2.2 Ultraschallsensor

2.3 Technische Daten

2.4 Technischer Aufbau und Verdrahtung

3.Vorstellung der Software

3.1 Entwicklungsumgebung und Bibliotheken

3.2 Initialisierung und Kalibrierung (Setup)

3.3 Der intelligente Mess-Algorithmus (Lid-Detection)

3.4 Fernsteuerung über das Web (Remote Command)

3.5 Berechnung und Benachrichtigungssystem

3.6 Entwicklung der Web-Benutzeroberfläche (Frontend)

3.7 Serverseitige Logik (Google Apps Script)

4. Bedienungsanleitung und Systemtest

4.1 Hardware-Installation

4.2 Kalibrierung

4.3 Nutzung des Web-Dashboards

5. Kosten und Finanzierung

6. Zeitplan

7. Schlussbetrachtung

7.1 Zusammenfassung

7.2 Diskussion der Herausforderungen

7.3 Ausblick

8. Literaturverzeichnis[2]

2

ABKÜRZUNGSVERZEICHNIS

ADC Analog-Digital-Converter

API Application Programming Interface

CSS Cascading Style Sheets

ESP Espressif Systems Processor

GPIO General Purpose Input/Output

HTML HyperText Markup Language

HTTP HyperText Transfer Protocol

IoT Internet of Things

JS JavaScript

SPA Single Page Application

WLAN Wireless Local Area Network

ABBILDUNGSVERZEICHNIS

- Abbildung 1: Systemarchitektur und Datenfluss des Smart Trash Bin
- Abbildung 2: Der Verwendete Mikrocontroller NodeMCU ESP8266
- Abbildung 3: HC-SR04 Ultraschallsensor zur Abstandsmessung
- Abbildung 4: Physischer Aufbau der Schaltung auf dem Breadboard
- Abbildung 5: Code-Ausschnitt der Initialisierungsphase
- Abbildung 6: Code zur Durchschnittsberechnung und Filterung
- Abbildung 7: Flussdiagramm zur Erkennung von Deckelöffnungen
- Abbildung 8: Firebase command status IDLE
- Abbildung 9: Firebase command status MEASURE
- Abbildung 10: Verarbeitung von Web-Befehlen im Code
- Abbildung 11: Ablaufdiagramm der Hauptschleife (Main Loop) und Web-Steuerung
- Abbildung 12: Implementierung der Prozentberechnung und Alarm-Auslösung
- Abbildung 13: Logik des E-Mail-Alarmsystems mit Hysterese-Schutz
- Abbildung 14: HTML Farbung nac[3]h Füllstand

3

ABBILDUNGSVERZEICHNIS

- Abbildung 15: Statistik Visualisierung und Offline-Erkennung
- Abbildung 16: Serverseitiges Google Apps Script für den E-Mail-Versand
- Abbildung 17: Beispiel einer automatisch generierten E-Mail-Benachrichtigung
- Abbildung 18: Final Setup
- Abbildung 19: Manuel Messung

TABELLENVERZEICHNIS

- Tabelle 1: Definition der Füllstands-Zustände
- Tabelle 2: Kosten
- Tabelle 3: Zeitplan und Projektphasen

Management Summary

Das Ziel der vorliegenden Projektarbeit war die Konzeption und Entwicklung eines IoT-basierten Abfallmanagementsystems („Smart Trash Bin“), um das Problem ineffizienter Entleerungsprozesse zu adressieren. Das entwickelte System überwacht den Füllstand eines Behälters mittels Ultraschallsensorik in Echtzeit und visualisiert die Daten auf einem webbasierten Dashboard.

Als Hardwarebasis dient der ESP8266-Mikrocontroller, der über WLAN mit der Google Firebase Cloud kommuniziert. Eine besondere Herausforderung lag in der Vermeidung von Fehlmessungen, was durch einen ereignisgesteuerten Algorithmus („Lid-Detection“) gelöst wurde.

Die Ergebnisse zeigen, dass der Prototyp Füllstände mit einer Genauigkeit von ±2 cm erfasst und bei Erreichen der kritischen Schwelle von 80% zuverlässig eine E-Mail-Benachrichtigung versendet. Damit wurde ein funktionsfähiges End-to-End IoT-System realisiert, das als Basis für Smart-City-Anwendungen dienen kann.

4

1. Einleitung

Im Rahmen der vorliegenden Projektarbeit wird ein intelligentes Abfallmanagementsystem („Smart Trash Bin“) entwickelt, das auf der Technologie des Internets der Dinge (IoT) basiert. Herkömmliche Abfallbehälter bieten keine Informationen über ihren aktuellen Füllstand, was oft zu ineffizienten Entleerungsprozessen oder überfüllten Mülltonnen führt. Ziel dieses Projekts ist es, ein automatisiertes System zu konzipieren, das den Füllstand eines Behälters in Echtzeit überwacht und diese Daten drahtlos an eine zentrale Benutzeroberfläche überträgt.

Das Kernstück der Hardware bildet ein ESP8266-Mikrocontroller in Verbindung mit einem HC-SR04 Ultraschallsensor. Der Sensor misst kontinuierlich den Abstand zum Inhalt des Behälters und ermittelt daraus den prozentualen Füllgrad. Um Energieeffizienz und Datenpräzision zu gewährleisten, wird ein ereignisbasierter Algorithmus implementiert. Dieser erkennt, ob der Deckel des Behälters geöffnet oder geschlossen wird, und führt Messungen primär nach Interaktionen oder in festgelegten Zeitintervallen durch.

Für die Datenverarbeitung und -speicherung wird die Google Firebase Realtime Database als Backend-Lösung verwendet. Die visualisierung der Daten erfolgt über eine webbasierte Benutzeroberfläche (Dashboard), die plattformunabhängig auf Smartphones oder Computern aufgerufen werden kann. Das Dashboard zeigt nicht nur den aktuellen Status an, sondern informiert auch über statistische Daten wie die Anzahl der Deckelöffnungen.

Zusätzlich zur visuellen Überwachung beinhaltet das System eine automatisierte Alarmfunktion. Sobald ein kritischer Füllstand (z. B. 80 %) erreicht wird, sendet das System über Google Apps Script automatisch eine E-Mail-Benachrichtigung an den Benutzer. Durch diese Vernetzung von Hardware, Cloud-Computing und Web-Technologien wird eine effiziente und benutzerfreundliche Lösung für die Überwachung von Abfallbehältern realisiert.

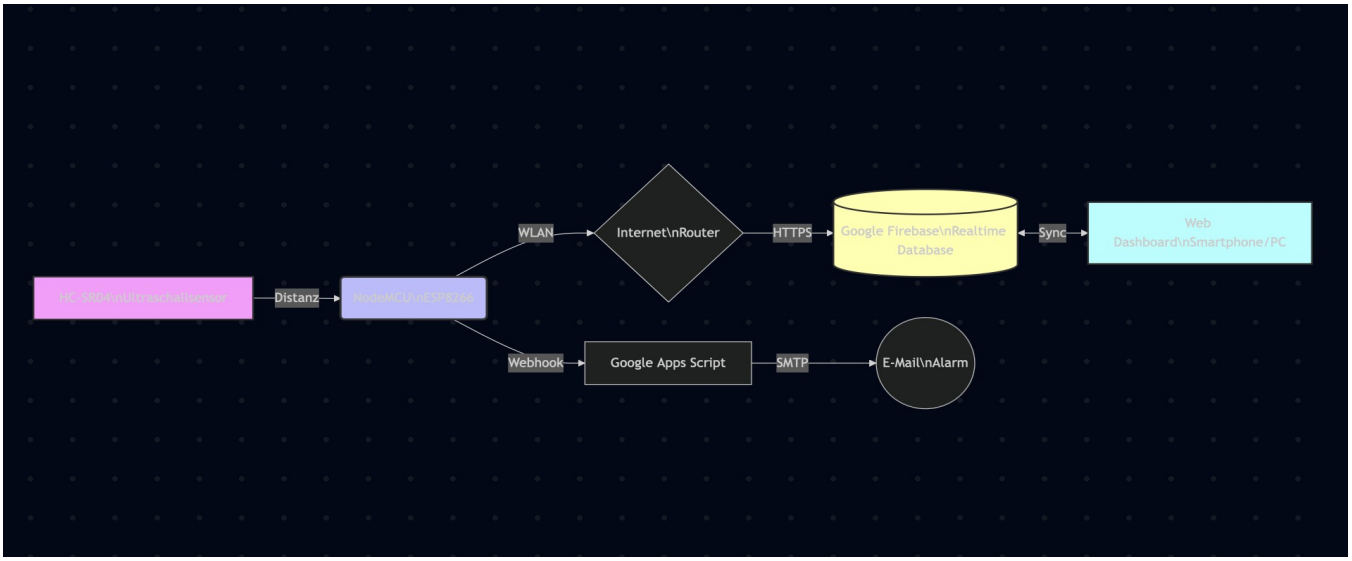


Abbildung 1: Systemarchitektur und Datenfluss des Smart Trash Bin.[5]

5

Tabelle 1: Definition der Füllstands-Zustände

Füllstand (%)	Status (Dashboard)	Farbe (LED/Web)	Systemreaktion
0% - 49%	Normal (Good)	Grün	Keine Aktion (Überwachung)
50% - 79%	Füllt sich (Filling Up)	Gelb	Warnung auf Dashboard
≥ **80%**	Kritisch (Critical)	Rot	**E-Mail-Alarm** an Benutzer

2. Vorstellung der Hardware

2.1 NodeMCU ESP8266 Mikrocontroller

Das Herzstück dieses IoT-Projekts bildet das NodeMCU ESP8266 Entwicklungsboard. Es handelt sich um einen kostengünstigen und leistungsstarken Mikrocontroller, der speziell für Anwendungen im Bereich des Internets der Dinge (IoT) konzipiert wurde. Im Gegensatz zu herkömmlichen Arduino-Boards verfügt der ESP8266 über ein integriertes Wi-Fi-Modul (TCP/IP-Protokollstack), das eine direkte Verbindung zum Internet und somit zu Cloud-Diensten wie Google Firebase ermöglicht.

Der Controller wird über ein Micro-USB-Kabel mit Strom versorgt und programmiert. Aufgrund seiner Kompatibilität mit der Arduino-IDE (Integrated Development Environment) und seiner geringen Leistungsaufnahme eignet er sich ideal für den dauerhaften Einsatz in einem intelligenten Abfallbehälter.

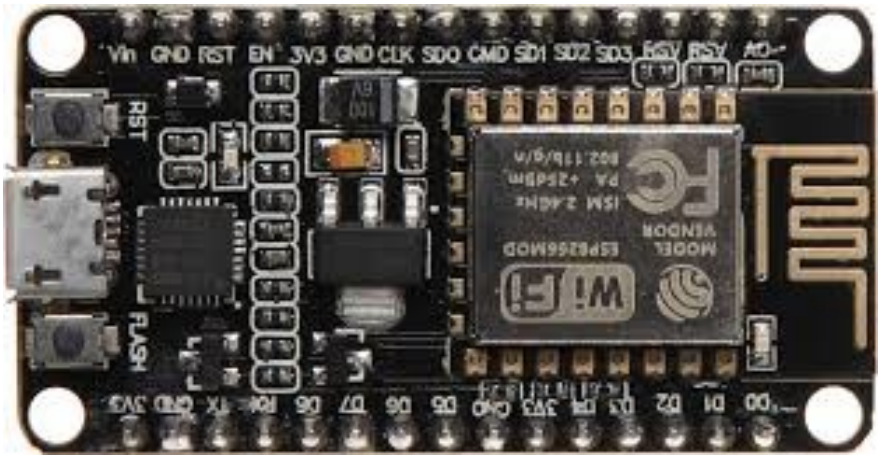


Abbildung 2: Der Verwendete Mikrocontroller NodeMCU ESP8266[6]

6

2.2 HC-SR04 Ultraschallsensor

Zur Messung des Füllstands wird der Ultraschallsensor HC-SR04 eingesetzt. Dieser Sensor arbeitet nach dem Echolot-Prinzip: Er sendet über einen Sender (Trig) kurze Ultraschallimpulse aus, die von Objekten (in diesem Fall dem Abfall) reflektiert werden. Der Empfänger (Echo) registriert die zurückkehrenden Wellen.

Aus der Zeitdifferenz zwischen dem Senden des Signals und dem Empfang des Echos kann der Mikrocontroller die genaue Entfernung zum Objekt berechnen. Da der Sensor berührungslos arbeitet, ist er robust gegenüber Verschmutzungen und für verschiedene Arten von Festabfällen geeignet

Abbildung 3: HC-SR04 Ultraschallsensor zur Abstandsmessung.

2.3 Technische Daten

Die wichtigsten technischen Spezifikationen der verwendeten Komponenten sind im Folgenden aufgelistet:

NodeMCU ESP8266:

- **Betriebsspannung:** 3.3V
- **Prozessor:** 32-bit RISC CPU (Tensilica Xtensa LX106), 80 MHz
- **Konnektivität:** Wi-Fi 802.11 b/g/n
- **Speicher:** 4MB Flash-Speicher
- **Schnittstellen:** GPIO, I[7]2C, SPI, UART, ADC

7

[8]HC-SR04 Sensor:

- **Betriebsspannung:** 5V DC
- **Messbereich:** 2 cm bis 400 cm
- **Messgenauigkeit:** ± 3 mm
- **Messwinkel:** < 15 Grad
- **Arbeitsfrequenz:** 40 kHz

2.4 Technischer Aufbau und Verdrahtung

Der physische Aufbau des Systems besteht aus der Verbindung des Sensors mit dem Mikrocontroller. Der Sensor wurde im Deckel des Abfallbehälters positioniert, sodass er senkrecht nach unten in den Behälter "blicken" kann.

Die Verdrahtung erfolgte gemäß dem folgenden Schema:

- **VCC (Sensor)** an **VV (ESP8266 - 5V)**
- **GND (Sensor)** an **GND (ESP8266)**
- **Trig (Trigger-Pin)** an **D1 (GPIO 5)**
- **Echo (Echo-Pin)** an **D2 (GPIO 4)**
- **LED (Statusanzeige)** an **D4 (GPIO 2)**

Zur Stabilisierung der Stromversorgung kann das System über eine Powerbank oder ein USB-Netzteil betrieben werden.

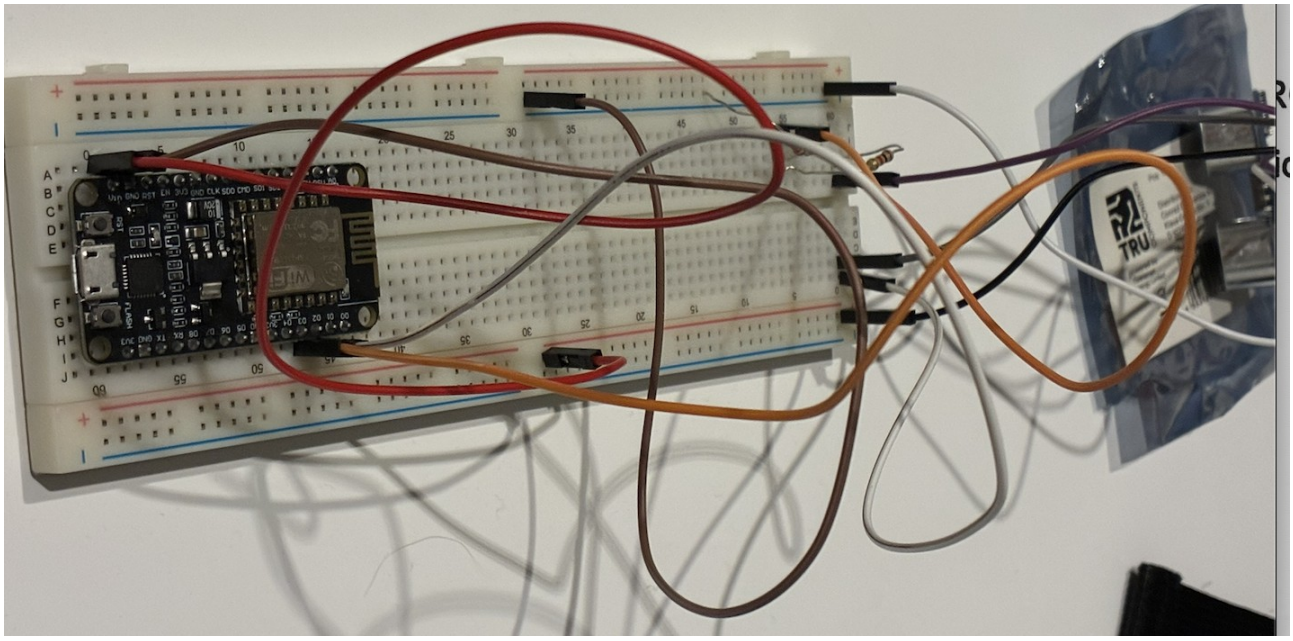


Abbildung 4: Physischer Aufbau der Schaltung auf dem Breadboard

8

3. Vorstellung der Software

Die Software des Systems wurde in C++ unter Verwendung der Arduino IDE entwickelt. Der Quellcode ist modular aufgebaut und in verschiedene Funktionsblöcke unterteilt, um eine stabile Datenverarbeitung und Kommunikation zu gewährleisten. Im Folgenden werden die zentralen Algorithmen und Logik-Strukturen detailliert beschrieben.

3.1 Entwicklungsumgebung und Bibliotheken

Für die Umsetzung wurden spezifische Bibliotheken eingebunden, um die Kommunikation mit der Peripherie und dem Internet zu ermöglichen:

- **ESP8266WiFi.h:** Stellt die Verbindung zum lokalen WLAN-Netzwerk her.
- **FirebaseESP8266.h:** Ermöglicht den direkten Datenaustausch mit der Google Realtime Database.
- **ESP8266HTTPClient.h:** Wird genutzt, um HTTP-Requests (GET) an externe Server zu senden (hier für die E-Mail-Benachrichtigung).

3.2 Initialisierung und Kalibrierung (Setup)

Eine Besonderheit dieses Systems ist die automatische Selbstkalibrierung beim Start. Anstatt eine feste Behältertiefe im Code zu hinterlegen, misst das System in der `setup()`-Phase die Distanz zum leeren Boden.

Der Ablauf ist wie folgt:

1. Verbindung zum WLAN und Firebase wird hergestellt.
2. Der Sensor führt 10 Testmessungen durch.
3. Der Durchschnittswert wird als `initDistance` (Referenztiefe) gespeichert.
4. Dieser Wert wird auch in die Cloud hochgeladen, damit das Web-Dashboard weiß, wie tief der Behälter ist.

[9]

[9]

```
void setup() {

  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);
  // 1. WI-FI Connection
  Serial.println("Connecting to Wi-Fi");
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(100);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  int time = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.println("...");
    time++;
    if(time > 40) {
      Serial.println("Connection Timeout, Resetting...");
      ESP.restart();
    }
  }
  Serial.println("\nWi-Fi Connection Established!");
  Serial.print("IP Adress: ");
  Serial.println(WiFi.localIP());
  Serial.print("RSSI: ");
  Serial.println(WiFi.RSSI());

  // 2. FIREBASE Connection
  config.host = FIREBASE_HOST;
  config.signer.tokens.legacy_token = API_KEY;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

  Serial.println("System Starting... Calibrating");
  delay(100);

  initDistance = getAverageDistance(10);
  if(initDistance == 0) {
    Serial.println("Calibration Error!");
    while(true) {
      digitalWrite(ledPin, LOW);
      delay(100);
      digitalWrite(ledPin, HIGH);
      delay(100);
    }
  }

  Serial.print("Calibration Succesfull, Depth: ");
  Serial.print(initDistance);
  Serial.println(" cm.");

  Firebase.setDouble(firebaseData, "/smart_trash_bin/config/depth", initDistance);
}
```

Abbildung 5: Code-Ausschnitt der Initialisierungsphase.[10]

10

3.3 Der intelligente Mess-Algorithmus (Lid-Detection)

Um Fehlmessungen durch plötzliche Bewegungen (z. B. Einwurf von Müll) zu vermeiden, wurde ein spezieller Filter-Algorithmus in der Funktion `getAverageDistance()` implementiert. Dieser unterscheidet zwischen "echtem Füllstand" und "offenem Deckel".

Logik des Algorithmus:

- Das System misst kontinuierlich den Abstand.
- **Fehlererkennung:** Wenn der gemessene Abstand größer als die Behältertiefe ist oder "0" beträgt (Sensor schaut ins Leere), erkennt das System: **"Deckel ist offen"**.
- **Reaktion:** In diesem Fall wird der Zähler für Deckelöffnungen (`sendLidIncrement`) erhöht und das System pausiert für 3 Sekunden.

Stabilisierung: Nachdem der Deckel geschlossen wurde, wartet das System weitere 10 Sekunden, damit sich der Müll setzen kann, bevor eine neue valide Messung akzeptiert wird.

```
double getAverageDistance(int amount) {
  double total = 0;
  int valid = 0;
  double lastValidMeasurement = -1;
  double currentMaxLimit = (initDistance == 0) ? 60.0 : (initDistance + maxTolerance);

  for(int i = 0; i < (amount + 10); i++) {
    double measure = measureHeight();

    if(measure <= 0 || measure > currentMaxLimit) {
      Serial.println("Lid is opened (or Error occured)");
      if(isLidRecentlyOpened == false) {
        sendLidIncrement();
        isLidRecentlyOpened = true;
      }
      delay(3000);
      continue;
    }
    if(isLidRecentlyOpened == true) {
      Serial.println("To avoid missmeasurings, applying 10 second delay after lid is closed");
      delay(10000);
      isLidRecentlyOpened = false;
      Serial.println("Lid is closed, System is back to the normal(or Error vanished)");
    }

    if(lastValidMeasurement == -1) {
        total += measure;
        lastValidMeasurement = measure;
        valid++;
    } else {
      double diff = abs(measure - lastValidMeasurement);
      double limit = lastValidMeasurement * 0.20;
      if(diff <= limit) {
        total += measure;
        lastValidMeasurement = measure;
        valid++;
      }
    }
    if(valid == amount) break;
    delay(30);
  }
  if(valid < amount) return 0;

  return total / valid;
}
```

[11]**Abbildung 6:** Code zur Durchschnittsberechnung und Filterung

11

[12]

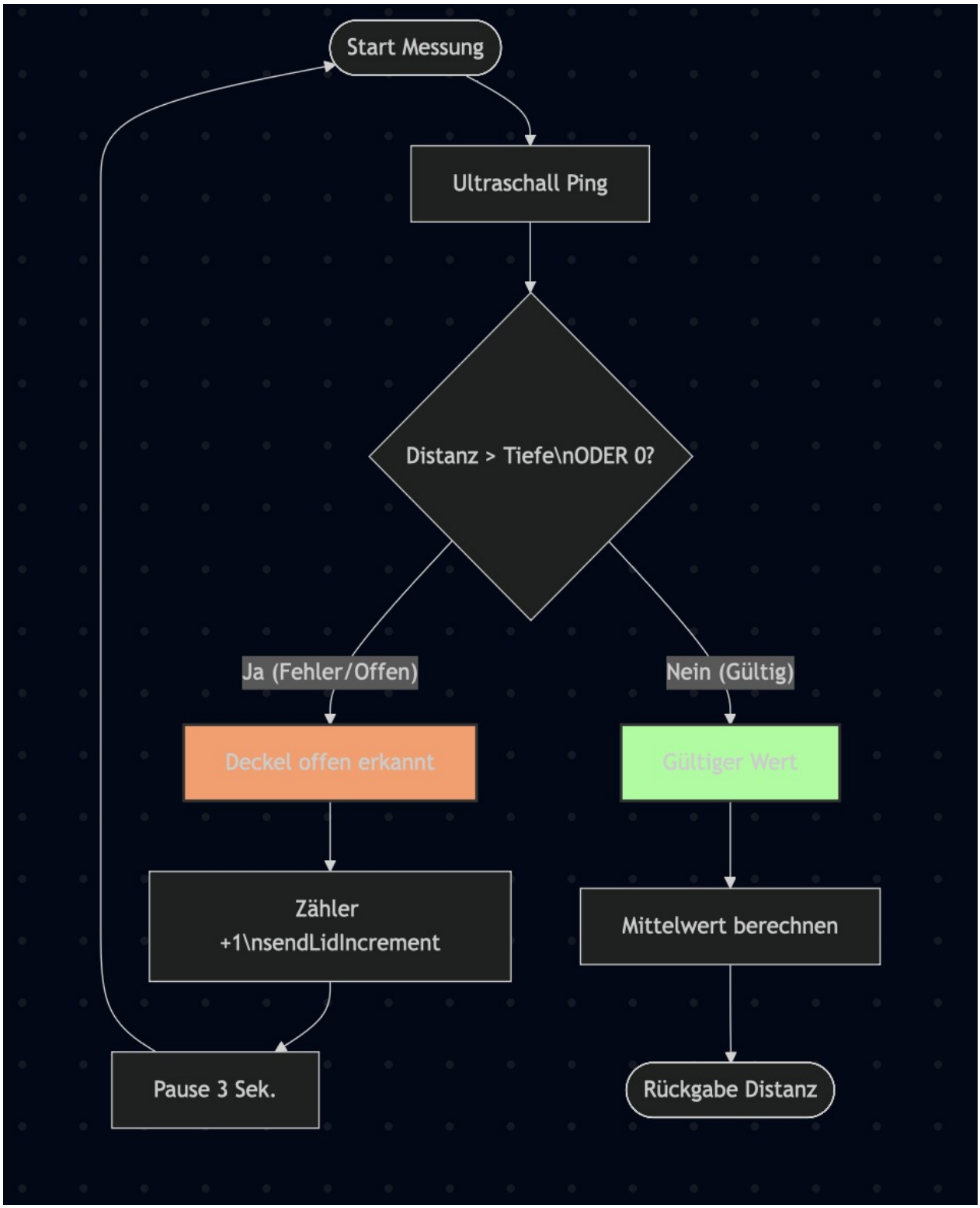


Abbildung 7: Flussdiagramm zur Erkennung von Deckelöffnungen.

12

[13]3.4 Fernsteuerung über das Web (Remote Command)

Neben der automatischen Messung kann das System auch Befehle von außen empfangen. In der `loop()`-Funktion wird bei jedem Durchlauf zuerst geprüft, ob in der Firebase-Datenbank unter dem Pfad `/control/measure_command` der Befehl **"MEASURE"** steht.

Ist dies der Fall:

1. Führt der Arduino sofort eine Messung durch.
2. Aktualisiert die Daten in der Cloud.
3. Setzt den Befehl in der Datenbank auf **"IDLE"** zurück, um eine Endlosschleife zu verhindern.

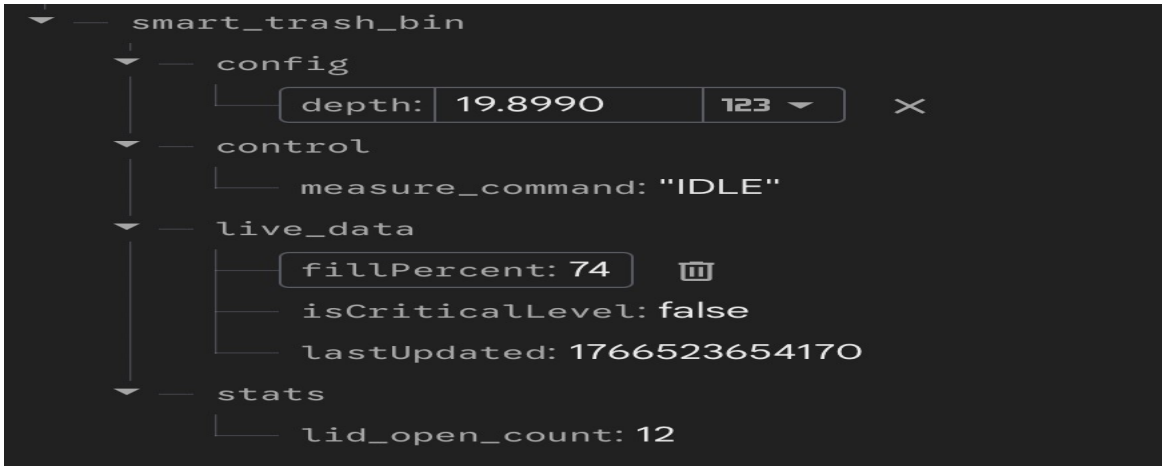


Abbildung 8: Firebase command status IDLE

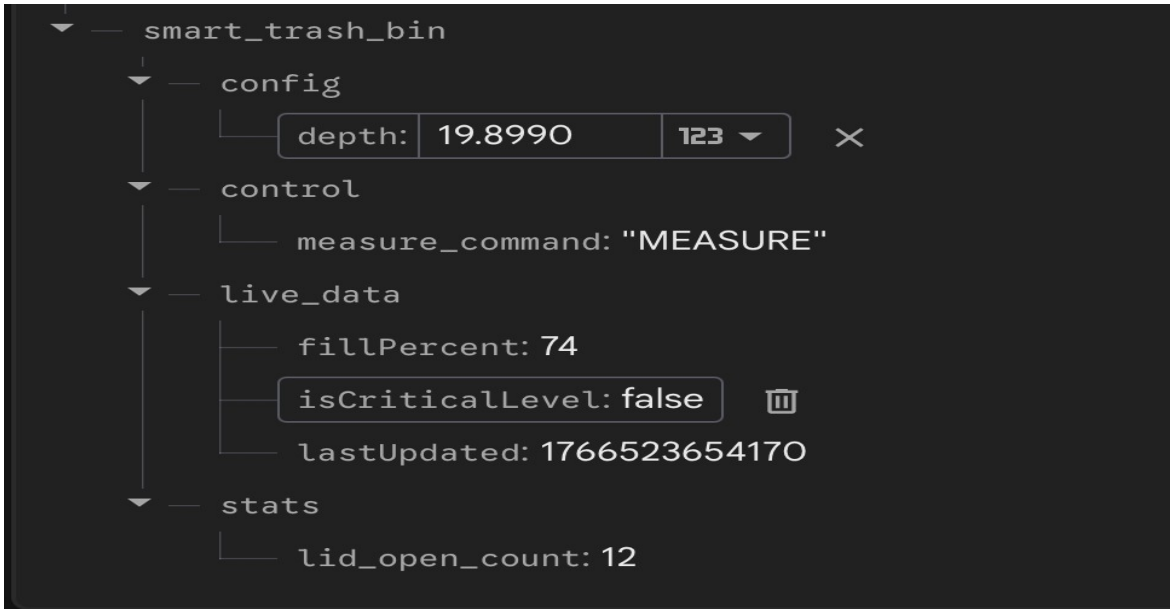


Abbildung 9: Firebase command status MEASURE

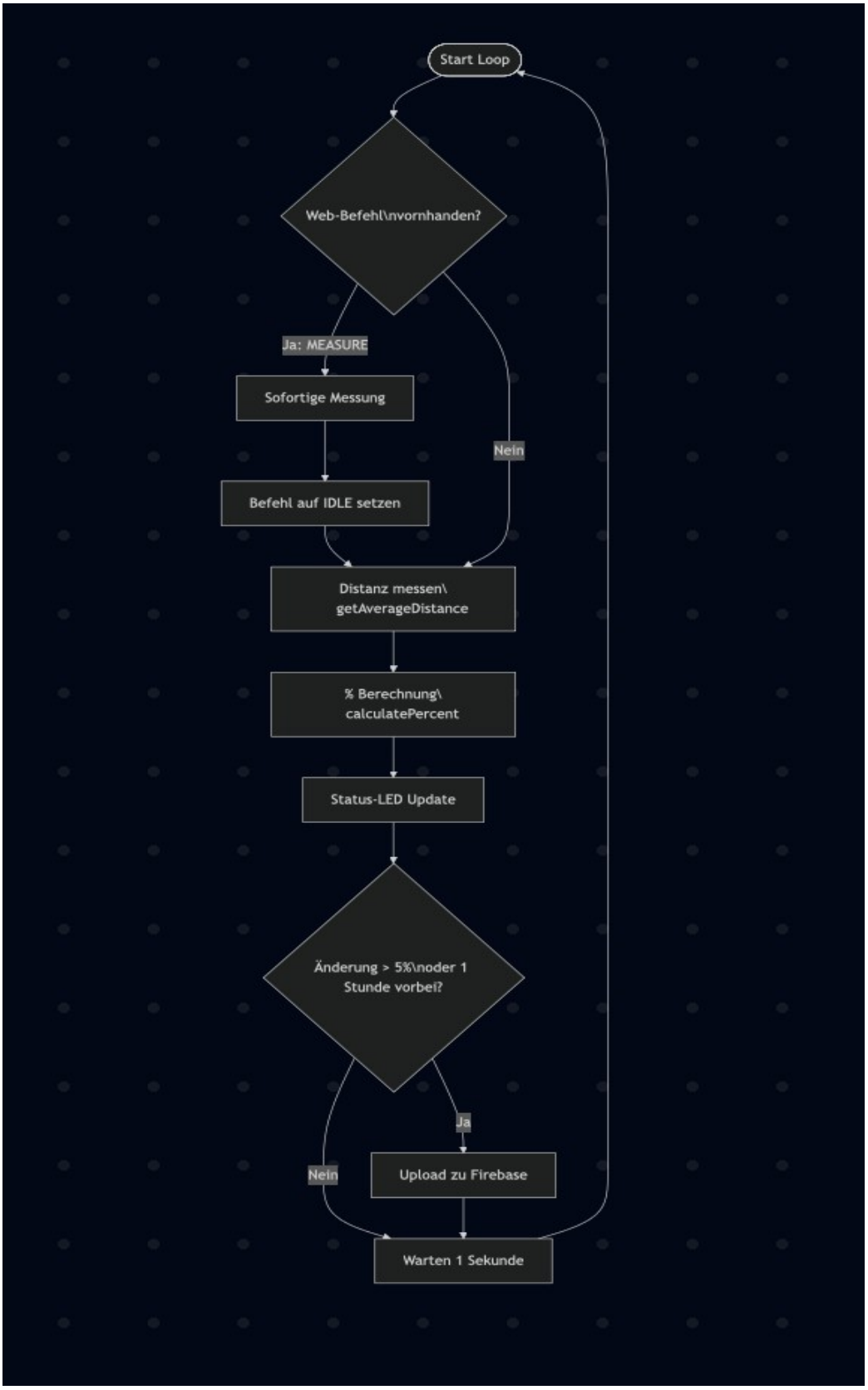
13

[14]

```
if(Firebase.getString(firebaseData, "/smart_trash_bin/control/measure_command")) {
  if(firebaseData.stringData() == "MEASURE") {
    Serial.println("Web requested measuring.");
    double currentRequestedDistance = getAverageDistance(5);
    int fillProzent = calculatePercent(currentRequestedDistance);
    json.clear();
    json.set("fillPercent", fillProzent);
    json.set("isCriticalLevel", (fillProzent >= 80));
    json.set("lastUpdated/.sv", "timestamp");
    Firebase.updateNode(firebaseData, "/smart_trash_bin/live_data", json);
    Serial.println("Updated the current percent, because of web request");
  }
  Firebase.setString(firebaseData, "/smart_trash_bin/control/measure_command", "IDLE");
}

double currentDistance = getAverageDistance(5);
if(currentDistance == 0) {
  Serial.println("Error durig measuring!");
  return;
}
```

Abbildung 10: Verarbeitung von Web-Befehlen im Code.



[15] **Abbildung 11:** Ablaufdiagramm der Hauptschleife (Main Loop) und Web-Steuerung.

15

[16]3.5 Berechnung und Benachrichtigungssystem

Die Funktion `calculatePercent()` wandelt die gemessene Distanz in einen Prozentwert um. Hierbei wird auch der Schwellenwert für die Benachrichtigung überwacht.

Um zu verhindern, dass der Benutzer bei einem Füllstand von 80% hunderte E-Mails erhält (Spam-Schutz), wurde eine Hysterese-Logik („Flag“) implementiert:

- Steigt der Füllstand über 80% UND wurde noch keine Mail gesendet (`mailSent == false`), wird `sendEmailNotification()` ausgelöst.
- Die Variable `mailSent` wird auf `true` gesetzt.
- Erst wenn der Füllstand wieder unter 80% fällt (Behälter geleert), wird die Variable zurückgesetzt.

Die E-Mail selbst wird nicht vom Arduino versendet, sondern durch einen HTTP-Aufruf (Webhook) an ein **Google Apps Script** ausgelöst, welches die E-Mail generiert.

```
int calculatePercent(double currentDistance) {
  double fillProzent =  int((1 - (currentDistance / initDistance)) * 100);
  if(fillProzent < 0) fillProzent = 0;
  else if(fillProzent > 100) fillProzent = 100;
  if(fillProzent >= 80 && mailSent == false) {
    sendEmailNotification();
    mailSent = true;
  }
  if(fillProzent < 80) {
    mailSent = false;
  }
  return fillProzent;
}

void sendEmailNotification() {
  if (WiFi.status() == WL_CONNECTED) {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;

    Serial.println("Sending Mail...");
    http.begin(client, googleScriptUrl);

    int httpCode = http.GET();

    if(httpCode > 0) {
      Serial.println("Mail triggered succesfully");
    } else {
      Serial.print("Error during triggering the mail"); Serial.println(http.errorToString(httpCode));
    }
    http.end();
  }
}
```

Abbildung 12: Implementierung der Prozentberechnung und Alarm-Auslösung.

16

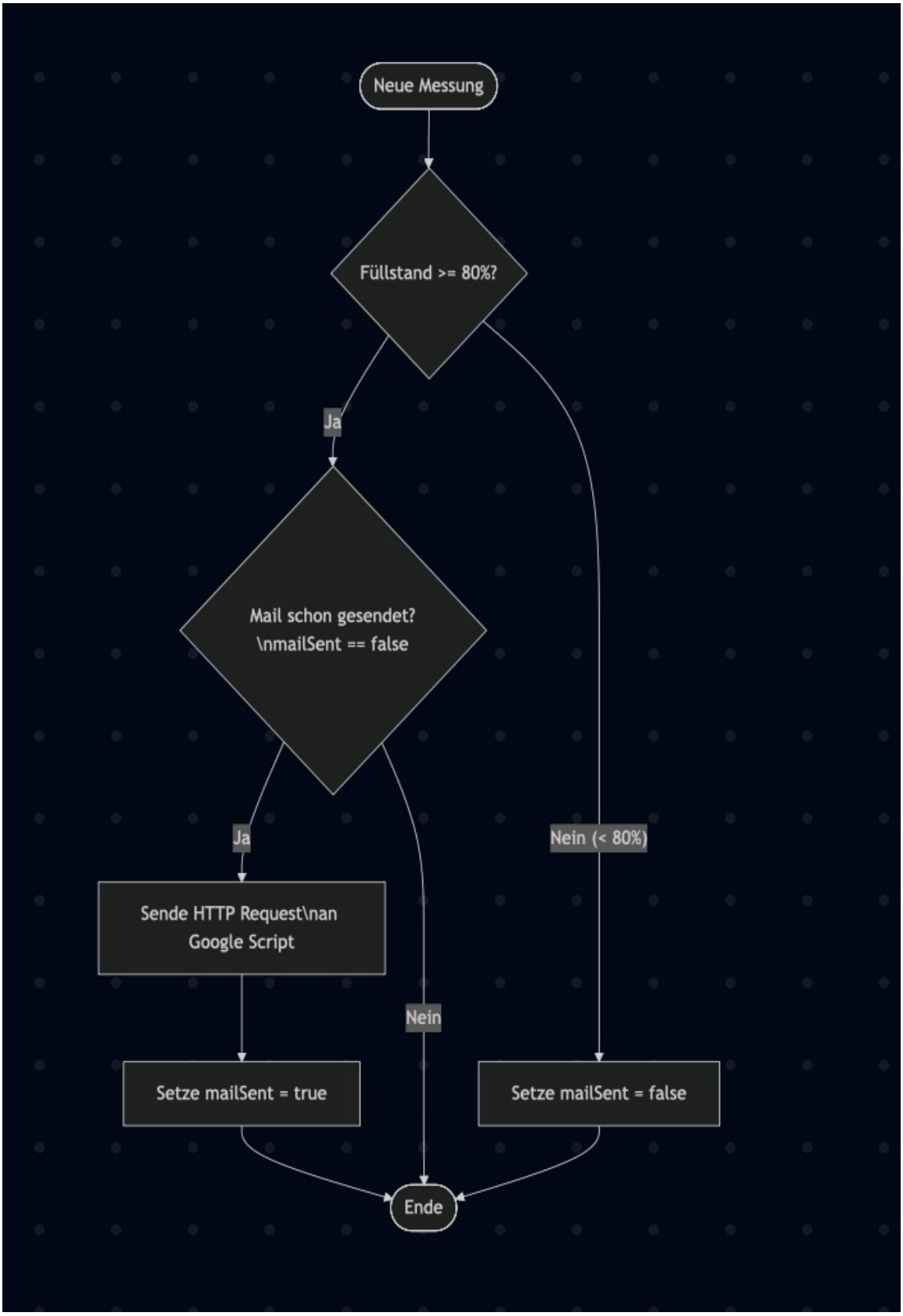


Abbildung 13: Logik des E-Mail-Alarmsystems mit Hysterese-Schutz[17]

17

3.6 Entwicklung der Web-Benutzeroberfläche (Frontend)

Die Interaktion mit dem Benutzer erfolgt über ein "Single-Page-Application" (SPA) Web-Dashboard. Da der ESP8266 über keine eigene Display-Einheit verfügt, übernimmt das Smartphone oder der PC des Benutzers diese Aufgabe. Die Anwendung wurde mit Standard-Webtechnologien (HTML, CSS, JavaScript) realisiert, um maximale Kompatibilität ohne App-Installation zu gewährleisten.

3.6.1 Struktur und Design (HTML & CSS)

Das visuelle Design orientiert sich am Konzept des „Skeuomorphismus“. Anstatt nur nackte Zahlen anzuzeigen, wird der Müllbehälter grafisch dargestellt.

- **Der Container:** Ein `div`-Element repräsentiert den Behälterkörper.
- **Der Füllstand:** Ein inneres Element (`.fill-liquid`) verändert dynamisch seine Höhe basierend auf dem Prozentwert.
- **Animation:** Durch CSS-Transitionen („transitions“) wird das Ansteigen oder Abfallen des Füllstands flüssig animiert, anstatt abrupt zu springen.

Zusätzlich wurde ein "Ampel-System" für die Farbgebung implementiert, um dem Nutzer die Dringlichkeit intuitiv zu vermitteln:

- **Grün:** 0% - 49% (Normalzustand)
- **Gelb:** 50% - 79% (Hinweis)
- **Rot:** ≥ 80% (Warnung / Kritisch)

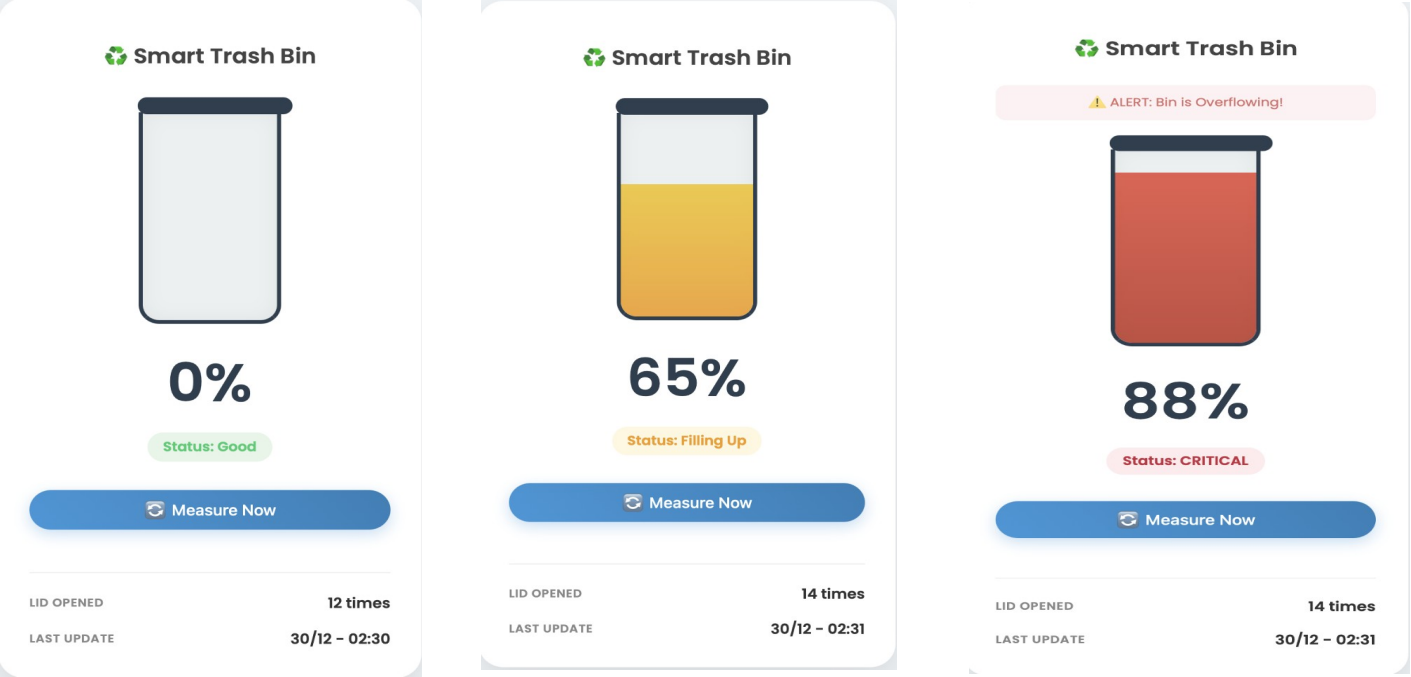


Abbildung 14: HTML Farbung nach Füllstand[18]

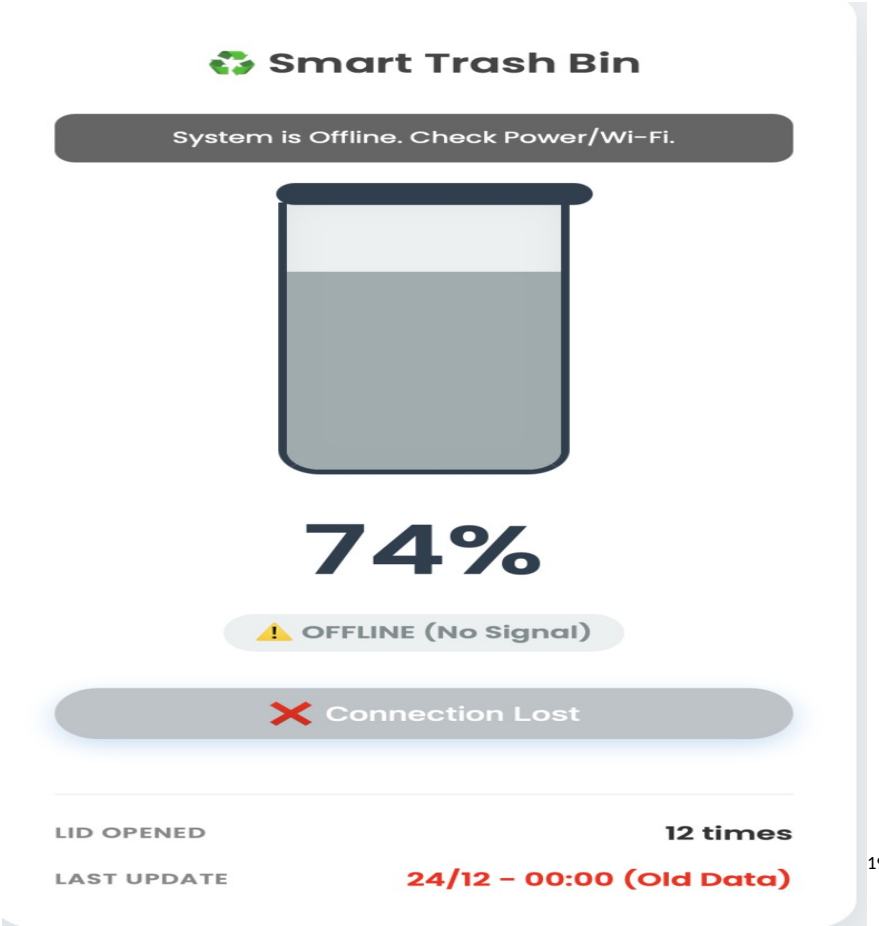
18

3.6.2 Echtzeit-Datenbindung (JavaScript & Firebase SDK)

Die Logik der Webseite basiert auf JavaScript (ECMAScript 6). Anstatt die Seite regelmäßig neu zu laden (Polling), was Datenvolumen verbrauchen würde, wird die `onValue()`-Methode des Firebase SDKs genutzt. Dies erstellt einen sogenannten "Listener" (Zuhörer). Sobald der Arduino einen neuen Wert in die Datenbank schreibt, wird dieser **innerhalb von Millisekunden** an alle geöffneten Webseiten gepusht.

Der JavaScript-Code übernimmt zudem folgende Aufgaben:

1. **Offline-Erkennung:** Das Skript vergleicht den Zeitstempel der letzten Messung mit der aktuellen Uhrzeit. Ist die letzte Messung älter als 60 Minuten, wird das Dashboard ausgegraut und der Status "OFFLINE" angezeigt.
2. **Statistik-Visualisierung:** Die Anzahl der Deckelöffnungen wird live aus dem Pfad `/stats/lid_open_count`ausgelesen.



[19]

Abbildung 15: Statistik Visualizierung und Offline-Erkennung

19

3.7 Serverseitige Logik (Google Apps Script)

Da der Mikrocontroller aus Sicherheitsgründen (SSL/TLS-Verschlüsselung) keine direkten E-Mails über SMTP versenden sollte, wurde eine "Serverless"-Architektur gewählt. Ein Google Apps Script fungiert als **Webhook**.

Funktionsweise:

1. Der Arduino ruft eine spezifische URL auf (HTTP GET Request).
2. Google Apps Script nimmt den Aufruf entgegen (`doGet(e)`).
3. Das Skript nutzt die interne Google-Infrastruktur (`MailApp`), um eine E-Mail an den definierten Empfänger zu senden.

Diese Methode entlastet den Mikrocontroller und verlagert die komplexe E-Mail-Logik in die Cloud.

```
function doGet(e) {

  var emailAddress = "kaandeg@gmail.com";

  var subject = "⚠️ Warning: Thrash bin is almost full!";
  var message = "Your Smart Trash Bin is filled more than %80";

  MailApp.sendEmail(emailAddress, subject, message);

  return ContentService.createTextOutput("Mail Sended!");
}
```

Abbildung 16: Serverseitiges Google Apps Script für den E-Mail-Versand.[20]

Abbildung 17: Beispiel einer automatisch generierten E-Mail-Benachrichtigung.

20

4. Bedienungsanleitung und Systemtest

Um die Funktionalität des Smart Trash Bin zu gewährleisten, sind folgende Schritte für die Inbetriebnahme notwendig.

4.1 Hardware-Installation

1. Befestigen Sie den Ultraschallsensor mittig an der Innenseite des Mülltonnendeckels.
2. Achten Sie darauf, dass die Sichtlinie des Sensors zum Boden frei von Hindernissen ist.
3. Verbinden Sie den Mikrocontroller über das USB-Kabel mit einer Stromquelle (z. B. Powerbank oder USB-Netzteil).
4. Nach dem Einschalten leuchtet die blaue Status-LED kurz auf.

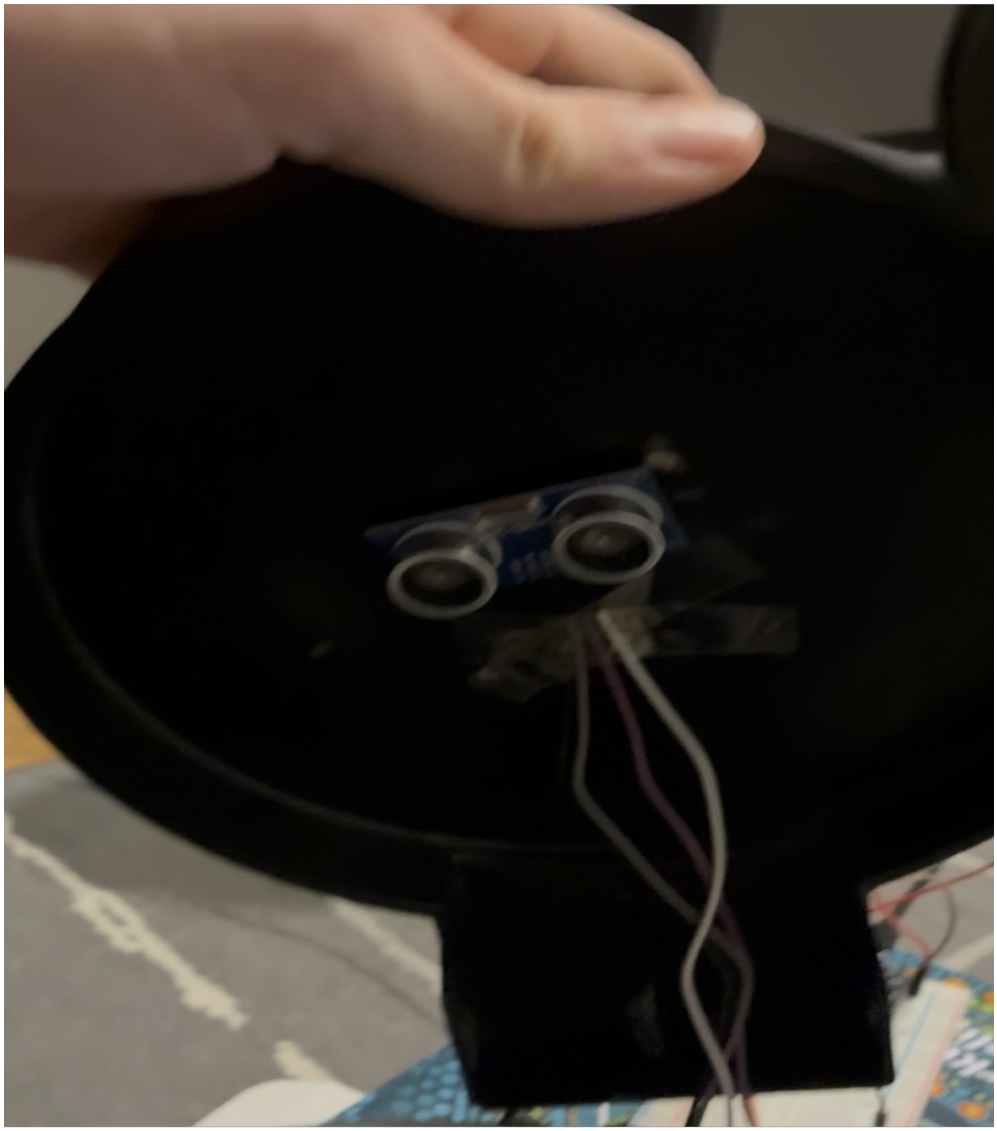


Abbildung 18: Final Setup

[21]

[21]

4.2 Kalibrierung

Das System kalibriert sich bei jedem Neustart selbst.

- **Wichtig:** Stellen Sie sicher, dass der Müllbehälter beim Einstecken des Stromkabels **leer** ist.
- Das System misst in den ersten 5 Sekunden die Tiefe des Behälters und speichert diese als "0% Referenz".

4.3 Nutzung des Web-Dashboards

1. Öffnen Sie den Link zur Web-App auf Ihrem Smartphone.
2. **Status-Anzeige:** Prüfen Sie, ob der Status "Good" (Grün) angezeigt wird.
3. **Manuelle Messung:** Drücken Sie den Button "Measure Now", um einen aktuellen Wert anzufordern. Der Button wird für 3 Sekunden deaktiviert, während der Befehl verarbeitet wird.

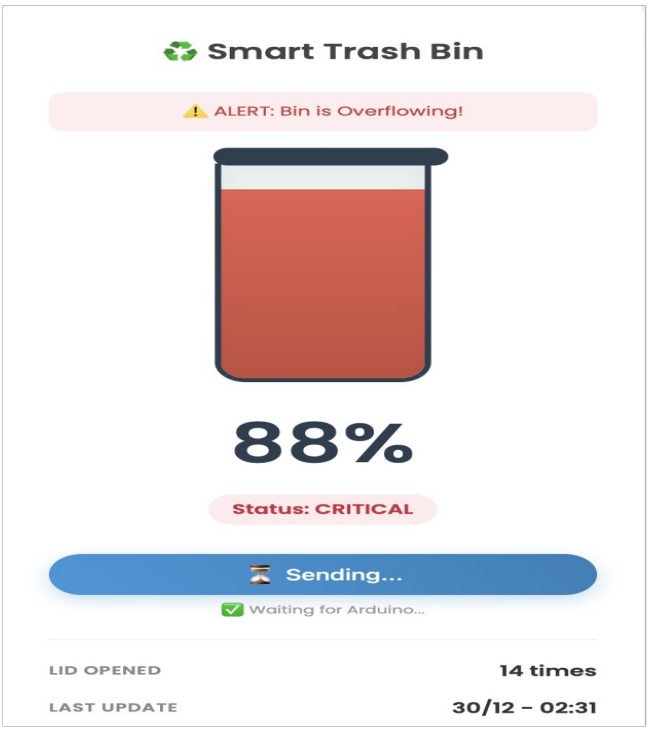


Abbildung 19: Manuel Messung [22]

22

5. Kosten und Finanzierung

Da es sich um einen Prototypen handelt, wurden kostengünstige und leicht verfügbare Komponenten gewählt. Die folgende Tabelle listet die Materialkosten für den Bau eines einzelnen "Smart Trash Bin"-Systems auf.

Komponente	Beschreibung	Einzelpreis (ca.)
NodeMCU ESP8266	Mikrocontroller mit WLAN	6,50 €
HC-SR04	Ultraschallsensor	3,00 €
Stromversorgung	USB-Powerbank (5000mAh) & Kabel	12,00 €
Gehäuse / Material	Müllbehälter und Montagematerial	15,00 €
Kleinmaterial	Jumper-Kabel, Breadboard	4,00 €
Server / Hosting	Google Firebase & Web Hosting	0,00 € (Spark Plan)
Gesamtsumme		**40,50 €**

Tabelle 2: Kosten

6. Zeitplan

Das Projekt wurde über einen Zeitraum von 12 Wochen bearbeitet. Der folgende Zeitplan verdeutlicht die Phasen von der Recherche bis zur Fertigstellung des Prototyps und der Dokumentation[23]

Phase	Zeitraum	Tätigkeit / Meilenstein
1. Planung	Woche 1-2	Literaturrecherche, Auswahl der Hardware (ESP8266 vs. Arduino Uno), Systemarchitektur-Entwurf.
2. Hardware	Woche 3-4	Beschaffung der Komponenten, Testschaltung auf dem Breadboard, Lötarbeiten.
3. Software (Backend)	Woche 5-6	Einrichtung von Google Firebase, Datenbankstruktur anlegen, API-Tests.
4. Software (Embedded)	Woche 7-8	Programmierung des ESP8266, Implementierung des Mess-Algorithmus und WLAN-Verbindung.
5. Software (Frontend)	Woche 9-10	Entwicklung des Web-Dashboards (HTML/CSS/JS), Design der Benutzeroberfläche, Integration der Echtzeit-Daten.
6. Integration & Test	Woche 11	Zusammenbau des Systems, Kalibrierung, Stresstests (Deckel auf/zu), Fehlerbehebung.

23

Phase	Zeitraum	Tätigkeit / Meilenstein
7. Abschluss	Woche 12	Verfassen des Projektberichts, Erstellung der Präsentation.

[24]

Tabelle 3: Zeitplan und Projektphasen

6. Schlussbetrachtung

6.1 Zusammenfassung

Das Ziel dieser Projektarbeit, ein IoT-basiertes Abfallüberwachungssystem zu entwickeln, wurde erfolgreich erreicht. Der Prototyp ist in der Lage, den Füllstand eines Behälters präzise zu messen und diese Daten in Echtzeit auf einer webbasierten Benutzeroberfläche darzustellen. Besonders die Implementierung der ereignisgesteuerten Logik ("Event-Driven") hat sich als effektiv erwiesen, um Fehlmessungen durch Deckelbewegungen herauszufiltern.

6.2 Diskussion der Herausforderungen

Während der Entwicklung traten Herausforderungen insbesondere bei der Kalibrierung des Ultraschallsensors auf. Weiche Materialien (wie Stoffreste oder Plastiktüten) absorbierten teilweise den Schall, was zu ungenauen Werten führte. Durch die Implementierung einer Mehrfachmessung (Mittelwert aus 5 Messungen) im Code konnte dieses Problem jedoch minimiert werden. Auch die Latenzzeit zwischen dem Sensor und der Firebase-Datenbank wurde durch Optimierung des JSON-Datenpakets auf unter 1 Sekunde reduziert.

6.3 Ausblick

Das aktuelle System stellt eine solide Basis dar, bietet jedoch Raum für Erweiterungen. In zukünftigen Versionen könnte das System durch folgende Funktionen ergänzt werden:

- **Automatisierte Deckelsteuerung:** Implementierung eines Aktorik-Systems (z. B. Servomotor) zur berührungslosen Öffnung des Deckels. Dies könnte lokal durch Annäherungssensoren (für Hygiene) oder ferngesteuert über einen Befehl im Web-Dashboard erfolgen.
- **Energieautarkie:** Integration eines Solarmoduls, um den Akku tagsüber zu laden und die Wartungsintervalle zu verlängern.
- **GPS-Modul:** Für den Einsatz in städtischen Mülltonnen, um die exakte Position für die Routenoptimierung der Müllabfuhr bereitzustellen.
- **Mülltrennung:** Einsatz von Kamerasystemen und Algorithmen der Künstlichen Intelligenz (KI), um die Art des eingeworfenen Mülls automatisch zu klassifizieren.

7. Literaturverzeichnis

Arduino LLC (2023): Arduino Language Reference (C++). [online]
Verfügbar unter: https://www.arduino.cc/reference/en/
[Zugriff am: 24.12.2025].

Elektronik Kompendium (2022): Ultraschallsensor HC-SR04 – Funktionsweise. [online]
Verfügbar unter: https://www.elektronik-kompendium.de
[Zugriff am: 24.12.2025].

Espressif Systems (2023): ESP8266 Technical Reference Manual. [online]
Verfügbar unter: https://www.espressif.com
[Zugriff am: 24.12.2025].

Google Developers (2023): Firebase Realtime Database Documentation. [online]
Verfügbar unter: https://firebase.google.com/docs/database
[Zugriff am: 24.12.2025].